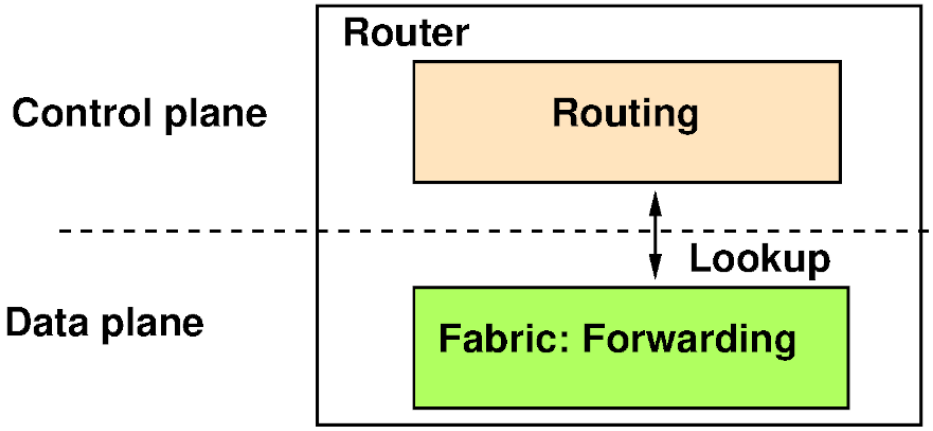
SDN – Software Defined Networking

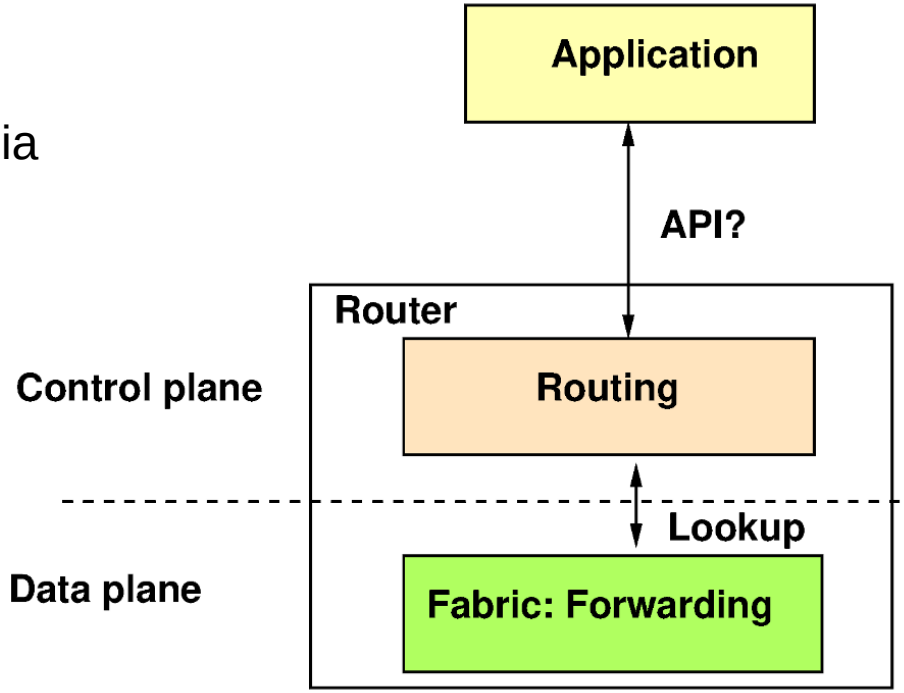
- Warstwa sterowania – *control plane*
  - wyznaczanie tras - *routing*
- Warstwa danych - *data plane*
  - macierz przełączająca – *fabric*
  - przekazywanie pakietów – *forwarding*
- Na styku warstw: wybór trasy pakietu – *lookup*



SDN – Software Defined Networking

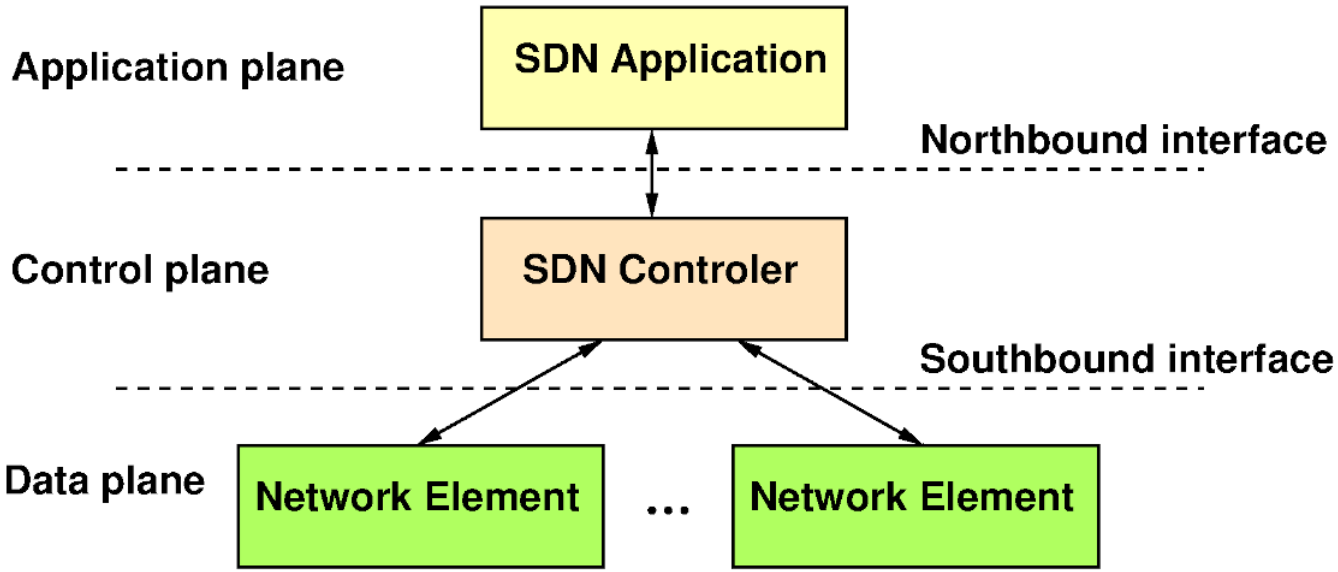
- Warstwa sterowania – *control plane*
  - wyznaczanie tras - *routing*
- Warstwa danych - *data plane*
  - macierz przełączająca – *fabric*
  - przekazywanie pakietów – *forwarding*
- Na styku warstw: wybór trasy pakietu – *lookup*

- Brak standardowego interfejsu do zarządzania



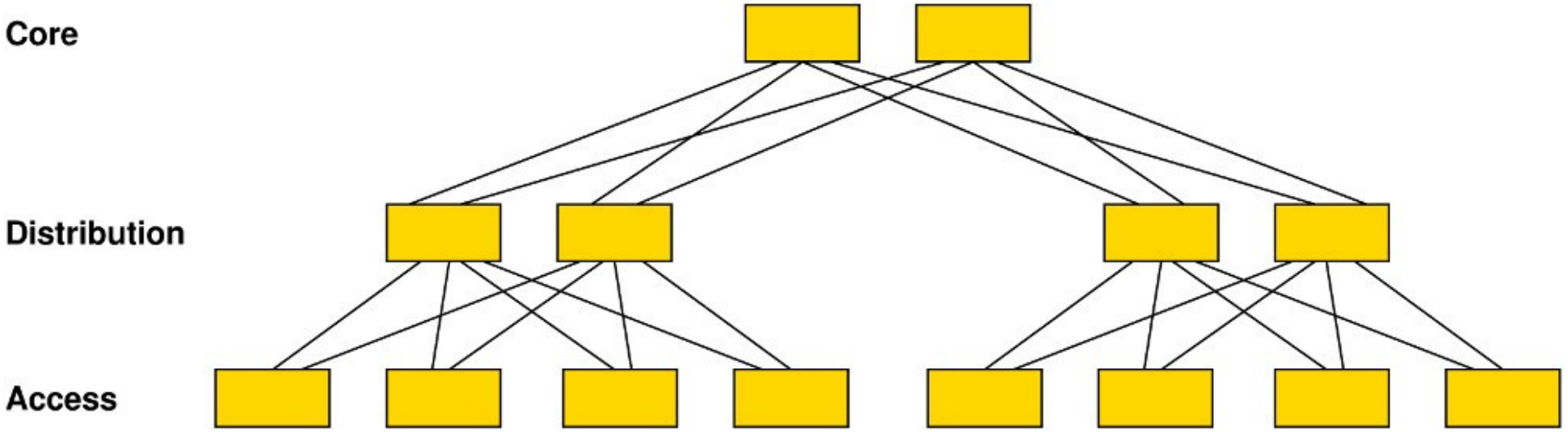
SDN – Software Defined Networking

- Kontroler konfiguruje urządzenia zgodnie z poleceniami aplikacji
- Urządzenia zajmują się wyłącznie przekazywaniem pakietów
- Standardowe interfejsy (np. *OpenFlow)*
- Budowa sieci nakładkowej *(overlay)* przy użyciu zasobów sieci podkładowej *(underlay)*
  - *L3: sieć IP, L2: vlany*
  - wykorzystanie tunelowania: VxLAN, Geneve, QinQ



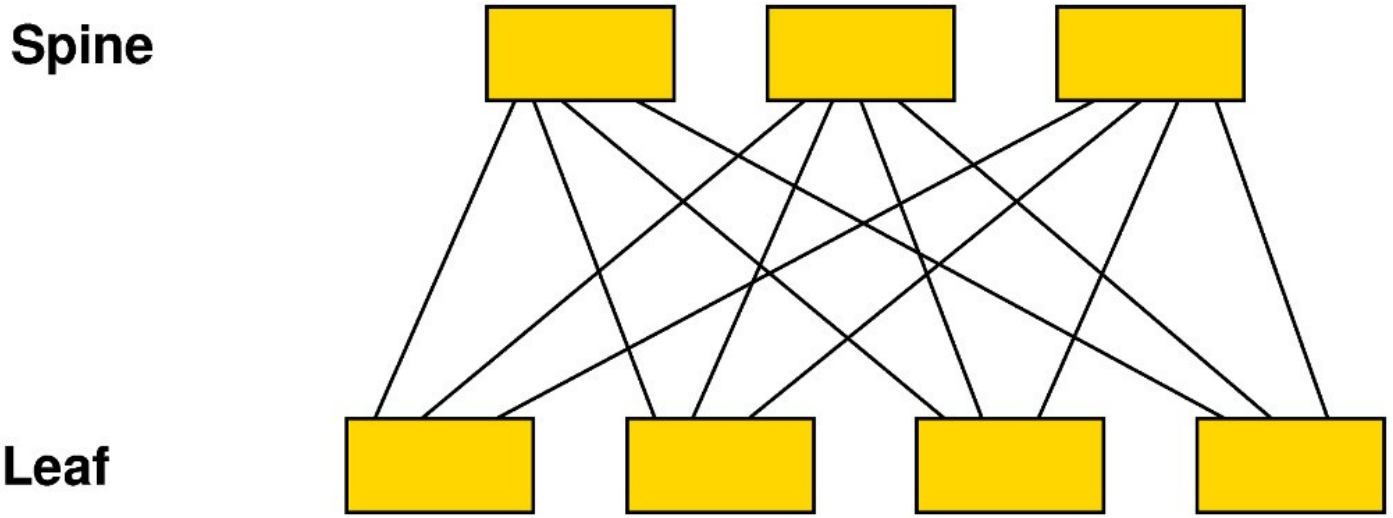
Budowa sieci overlay

- Topologia
  - pojedynczy przełącznik + vlany (L2)
  - core-disttribution-access + vlany (L2)
  - Leaf-spine + GENEVE/VxLAN (L3)
- L2 vs. L3
  - STP (spanning tree) vs. ECMP (*equal cost multipath*)
  - ruch północ-południe vs. wschód-zachód



Budowa sieci overlay

- Topologia
  - pojedynczy przełącznik + vlany (L2)
  - core-disttribution-access + vlany (L2)
  - Leaf-spine + GENEVE/VxLAN (L3)
- L2 vs. L3
  - STP (spanning tree) vs. ECMP (*equal cost multipath*)
  - ruch północ-południe vs. wschód-zachód

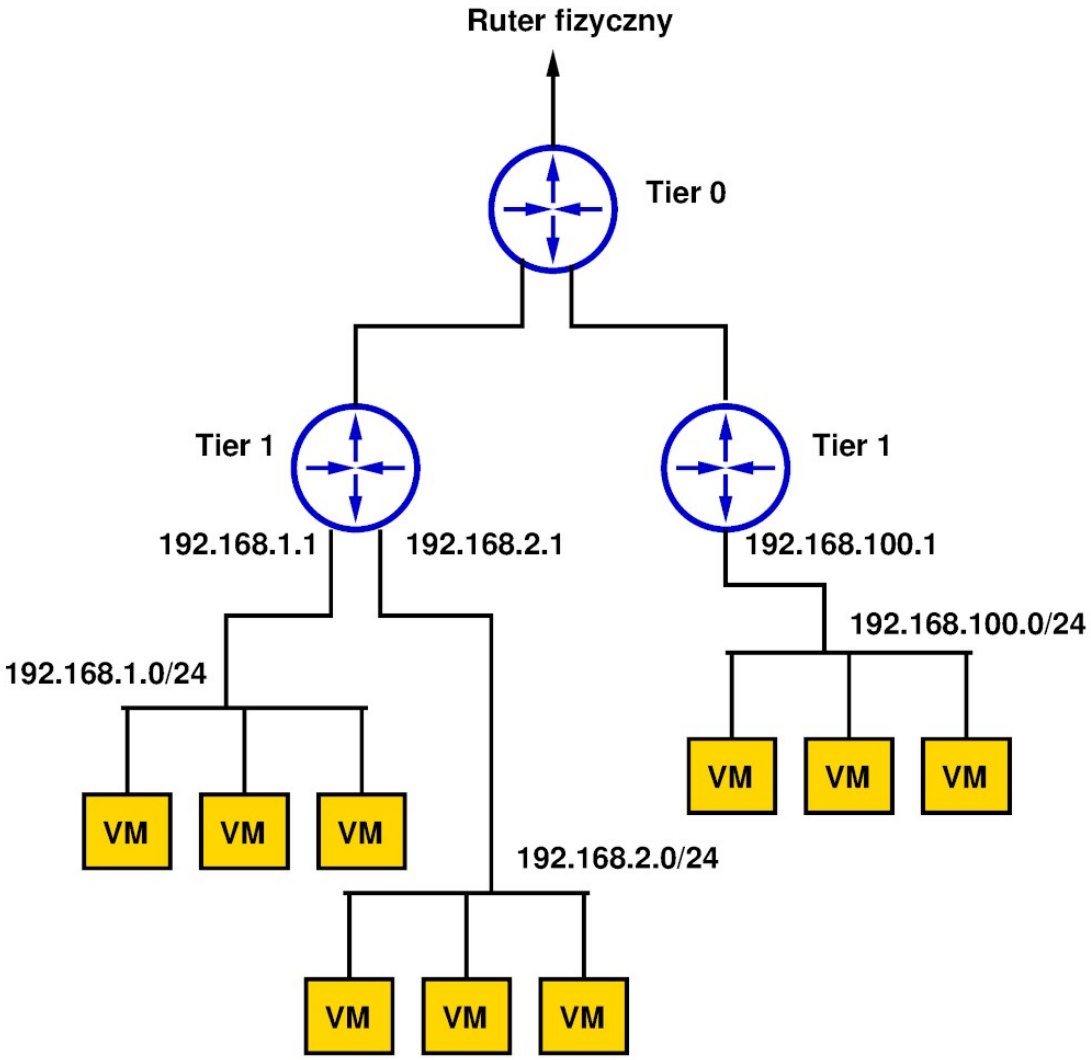


Tunele overlay

- VLAN – IEEE802.1q
  - Vlanid (tag) 12b – 4096 wartości
- QinQ – IEEE802.1ad
  - podwójne tagowanie
  - vlan transportowy
- VxLAN
  - VNI 24b – 16777216 wartości
  - nagłówek 8B
  - używa UDP
  - wsparcie w OpenVSwitch
- GENEVE – Generic Network Virtualization Encapsulation
  - rozszerzalny nagłówek zawiera opcje
  - wsparcie w OpenVSwitch
- Problem MTU, wskazane (konieczne) jumbo frame
  - wskazany *offloading*

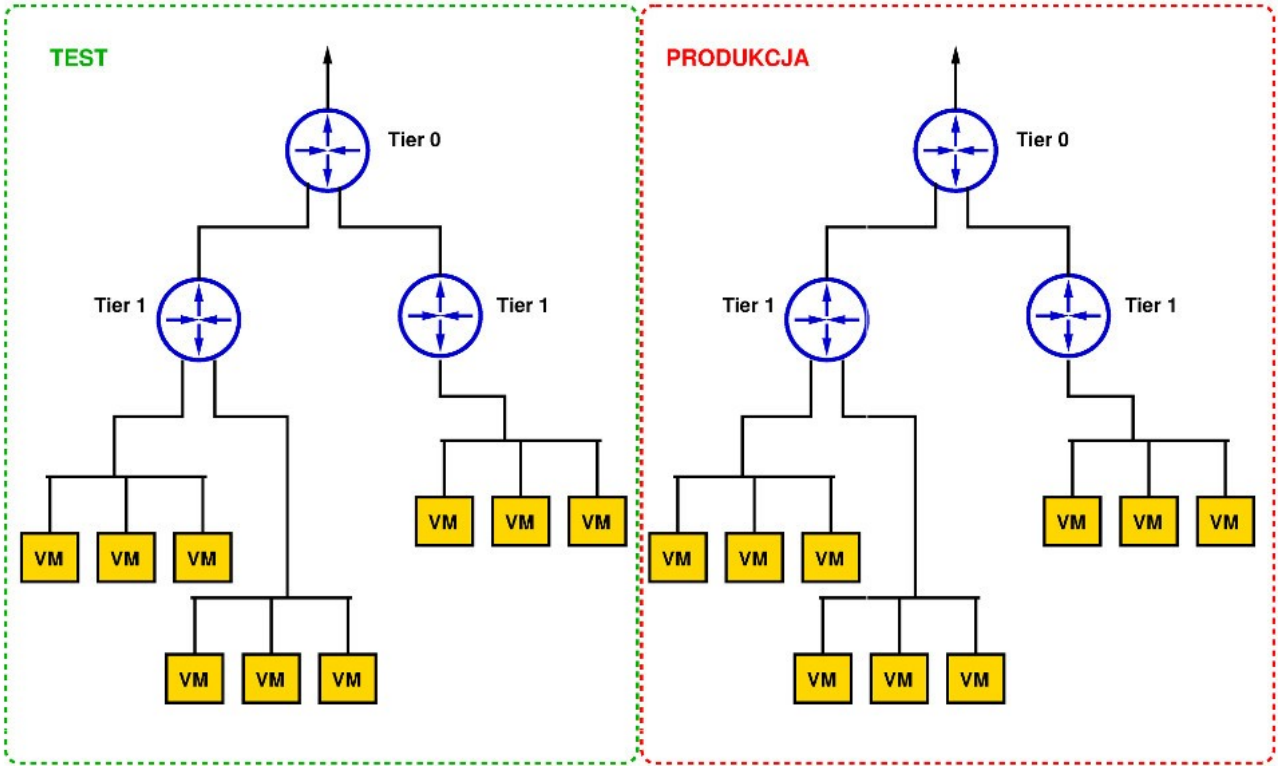
NSX-T architektura

- NSX Manager – centralny zarządca
  - control plane
  - dostarcza API i GUI
  - pracuje w klastrze HA
  - jeden adres IP
  - używa funkcji jądra ESXi (data plane)
- NSX Edge – lokalny agent
- Rutery
  - Tier 0 Gateway – łączność z siecią zewnętrzną
  - Tier 1 gateway – łączy segmenty
- Segmenty
  - Logiczne przełączniki dla VM (*portgroup*)
- Rozproszony firewall
  - Reguły na ruch z/do VM



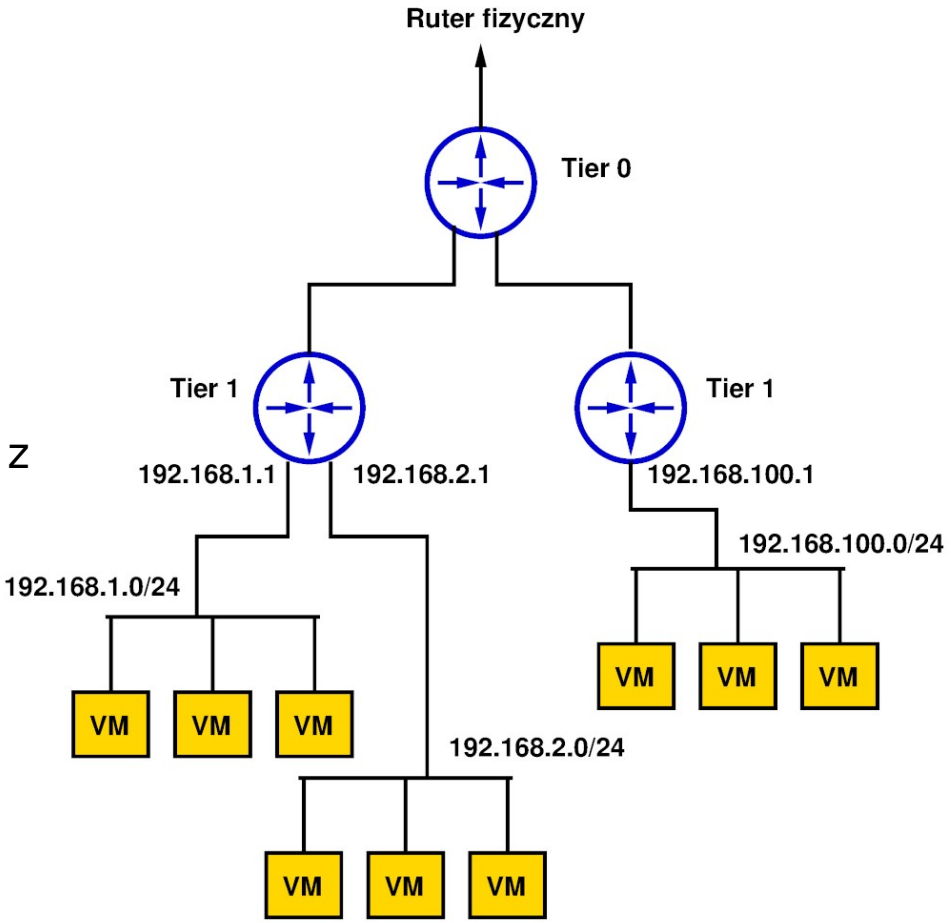
NSX-T topologia

- Hierarchia ruterów
  - tier1 – brama dla segmentów
  - tier0 – używa portów fizycznych
  - sieć podkładowa (*underlay*)
    - umożliwia rozpięcie tuneli GENEVE
  - protokoły rutingu: OSPF, **BGP**
    - polityki BGP
    - ECMP – equal cost multipath
  - reguły FW
- Rozproszony firewall (*distributed firewall*)
  - reguły chroniące VM
  - możliwa filtracja w segmencie

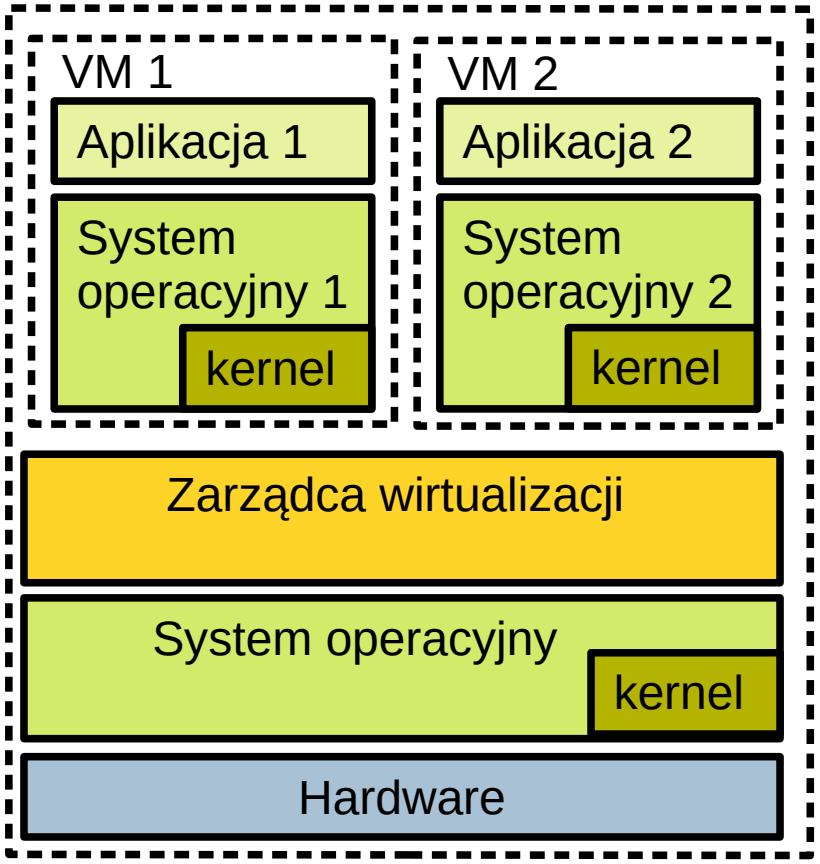


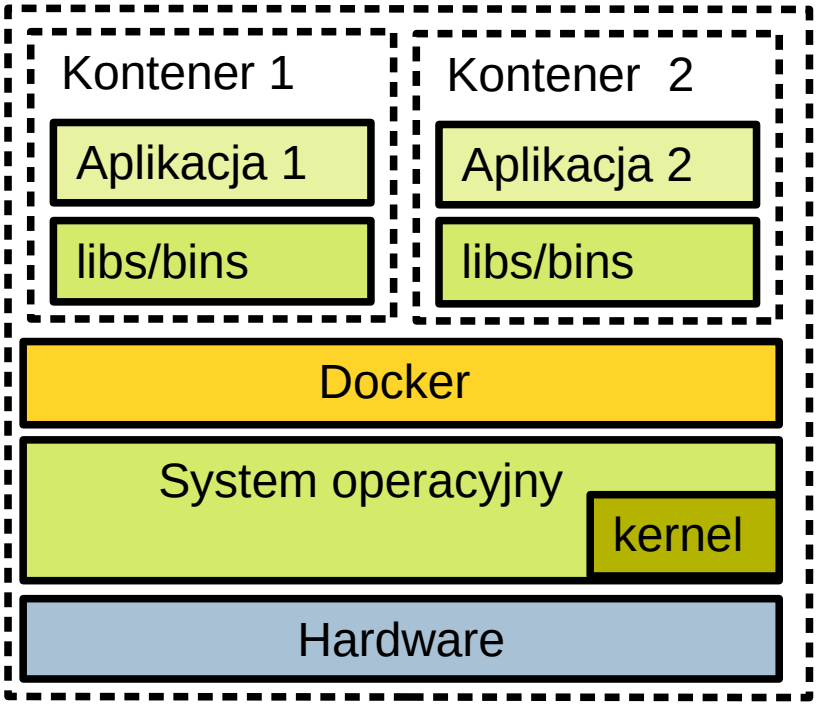
Dlaczego BGP

- RFC 4271 (z poprawkami)
- Trasowanie między systemami autonomicznymi
- Prywatne AS: 64512 – 65,534 (16b)
- Rutery w sąsiednich AS wymieniają trasy zgodnie z politykami:
  - export – ograniczenie rozsyłania prefiksów
  - import – ograniczenie przyjmowania tras do prefiksów
- Polityki określają rolę AS w stosunku do sąsiada:
  - Rozgłasza prefiksy == przyjmuje ruch do nich
  - Nie rozgłasza prefiksów == ukrywa sieci
  - Przyjmuje prefiksy == ruting przez rozgłaszającego



Konteneryzacja vs. wirtualizacja





- Lżejsza: wspólne jądro, tylko konieczne biblioteki i binaria
- Brak emulacji
- Łatwiejsze wdrożenie (*deployment*)
- Niższy poziom izolacji
  - przestrzeń nazw vs. oddzielny system operacyjny + emulowany sprzęt

Konteneryzacja

- Elementy:
  - Dockerfile – „przepis” na obraz kontenera
  - Obraz – szablon zawierający wszystkie elementy kontenera
  - Kontener – działająca realizacja obrazu
  - Docker hub – centralne repozytorium obrazów

Konteneryzacja

- Elementy:
  - Dockerfile – „przepis” na obraz kontenera
  - Obraz (*image*) – szablon zawierający wszystkie elementy kontenera
  - Kontener (*container*) – działająca realizacja obrazu
  - Docker hub – centralne repozytorium obrazów

  - analogia z OOP: obraz=klasa, kontener=obiekt

Konteneryzacja

- Elementy:
  - Dockerfile – „przepis” na obraz kontenera
  - Obraz (*image*) – szablon zawierający wszystkie elementy kontenera
  - Kontener (*container*) – działająca realizacja obrazu
  - Docker hub – centralne repozytorium obrazów

  - analogia z OOP: obraz=klasa, kontener=obiekt
- Obraz zawiera wszystkie biblioteki i binaria potrzebne do uruchomienia usługi/aplikacji
- Obraz może bazować na innym (prostszym) obrazie
  - analogia z OOP: dziedziczenie/rozszerzanie

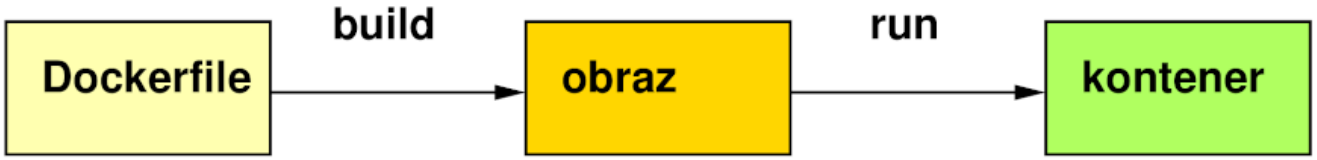
Konteneryzacja

- Elementy:
  - Dockerfile – „przepis” na obraz kontenera
  - Obraz (*image*) – szablon zawierający wszystkie elementy kontenera
  - Kontener (*container*) – działająca realizacja obrazu
  - Docker hub – centralne repozytorium obrazów

  - analogia z OOP: obraz=klasa, kontener=obiekt
- Obraz zawiera wszystkie biblioteki i binaria potrzebne do uruchomienia usługi/aplikacji
- Obraz może bazować na innym (prostszym) obrazie
  - analogia z OOP: dziedziczenie/rozszerzanie

- Łatwe wdrożenie
  - bazowanie na znanych obrazach
  - kontener jest kompletny
  - możliwość parametryzacji i skalowania/powielania

Konteneryzacja: Dockerfile




```
FROM ubuntu:latest

RUN apt-get update && apt-get install
-y nettools

WORKDIR /app

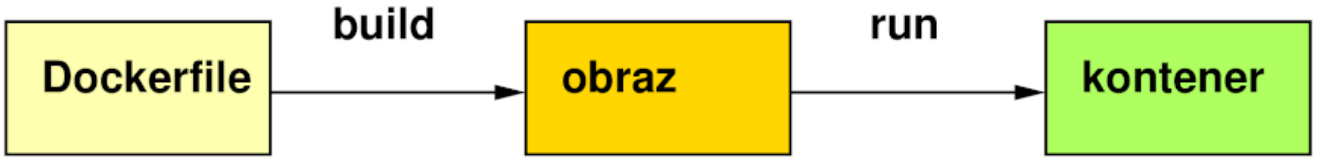
COPY .

ENTRYPOINT [ ”my_service”, ”runner” ]

CMD [ ”1” ]

EXPOSE 80/tcp
```

Konteneryzacja: Dockerfile




```
FROM ubuntu:latest

RUN apt-get update && apt-get install
-y nettools

WORKDIR /app

COPY .

ENTRYPOINT [ "my_service", "runner" ]

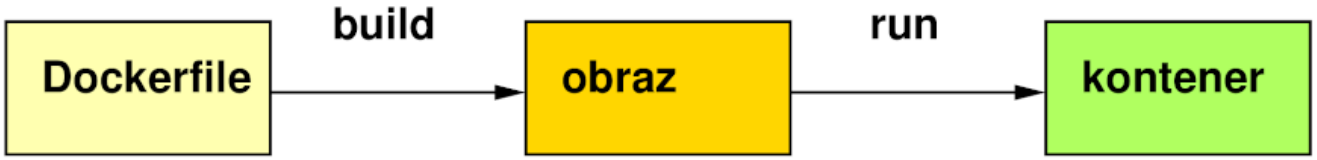
CMD [ "1" ]

EXPOSE 80/tcp
```

Obraz bazowy

Instalacja pakietów

Konteneryzacja: Dockerfile




```
FROM ubuntu:latest

RUN apt-get update && apt-get install
-y nettools

WORKDIR /app

COPY .

ENTRYPOINT [ "my_service", "runner" ]

CMD [ "1" ]

EXPOSE 80/tcp
```

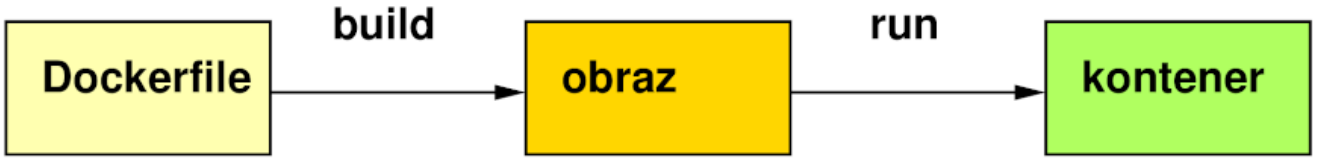
Obraz bazowy

Instalacja pakietów

Tworzy katalog i przechodzi do niego

Kopiuje zawartość bieżącego katalogu

Konteneryzacja: Dockerfile




```
FROM ubuntu:latest

RUN apt-get update && apt-get install
-y nettools

WORKDIR /app

COPY .

ENTRYPOINT [ "my_service", "runner" ]

CMD [ "1" ]

EXPOSE 80/tcp
```

Obraz bazowy

Instalacja pakietów

Tworzy katalog i przechodzi do niego

Kopiuje zawartość bieżącego katalogu

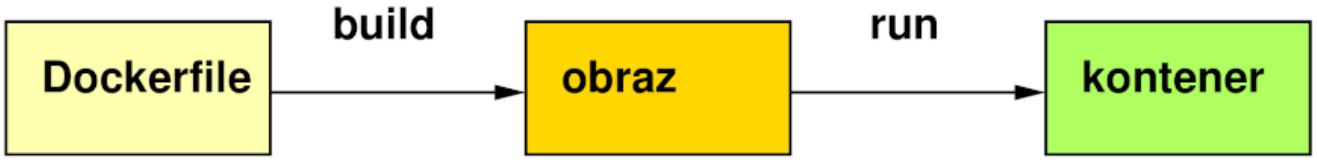
Uruchomienie usługi domyślne parametry*

Port, na którym nasłuchuje**

* Parametry można nadpisać przy uruchomieniu

** Port należy zamapować przy uruchomieniu – opcja -p

Konteneryzacja: Dockerfile




```
FROM ubuntu:latest

RUN apt-get update && apt-get
install -y nettools

WORKDIR /app

COPY .

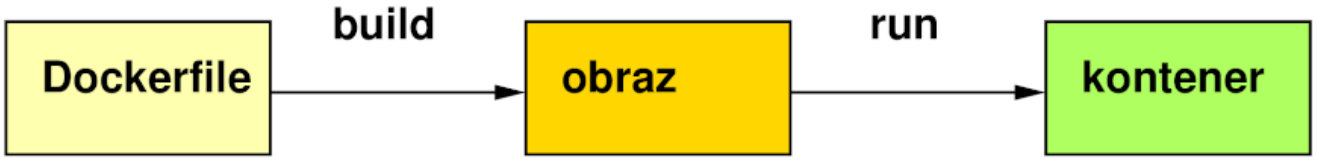
ENTRYPOINT [ "my_service" ]

CMD [ "1" ]

EXPOSE 80/tcp
```

- Warstwy
  - warstwa odpowiada linii w pliku
  - przechowywane do wykorzystania
    - budowane tylko zmienione
  - RO
- Przy stworzeniu kontenera (run) dodawana warstwa RW
  - przechowuje stan kontenera po wyłączeniu
  - możliwe kasowanie przy uruchomieniu (start)*

Konteneryzacja: Dockerfile




```
FROM ubuntu:latest

RUN apt-get update && apt-get
install -y nettools

WORKDIR /app

COPY .

ENTRYPOINT [ "my_service" ]

CMD [ "1" ]

EXPOSE 80/tcp
```

- Warstwy
  - warstwa odpowiada linii w pliku
  - przechowywane do wykorzystania
    - budowane tylko zmienione
  - RO
- Przy stworzeniu kontenera (run) dodawana warstwa RW
  - przechowuje stan kontenera po wyłączeniu
  - możliwe kasowanie przy uruchomieniu (start)*

* trwałość danych można zapewnić przez montowanie woluminów

Docker: sieć

- Tworzenie sieci: **`docker network create …`**
  - *driver* (**`--driver`**) definiuje typ sieci
- Dodawanie sieci przy tworzeniu kontenera: **`docker run … --network`** *name*
- Kontener można dołączyć i odłączyć od sieci *„w locie”*
  - **`docker network [connect|disconnect]`** *net_name container_name*

- ***Host networking***: brak izolacji od hosta
  - **`--network host`**

- **Bridge networking**:
  - domyślny (nie zalecany, lepiej stworzyć własne sieci)
  - **`--driver bridge`**
  - adresy ip z podanego zakresu,
  - **`--gateway`** ip dla rutera domyślnego dla kontenerów
  - możliwy nat (maskarada), mapowanie portów (**`-p 192.168.1.1:80:8888`**)

Docker: sieć

- ***Macvlan networking***: współdzieli wskazany interfejs (**`-parent`**)
  - widoczny adres mac kontenera
  - interfejs musi obsługiwać wiele adresów mac, ew. promiscuous
  - „omija docker L3 stack”
  - **`--gateway`** adres zewnętrznego rutera

- ***Ipvlan networking***: jak *macvlan*, ale omija problem wielu adresów mac/IP
  - Szybszy, prostszy stos

- ***Overlay networking***: sieć dla klastra